
Chaining Comparisons

Contents

- [Revision History](#)
- [Introduction](#)
 - [Existing Code in C++](#)
 - [Existing Code in Python](#)
- [Issues at Hand](#)
 - [Which operators can chain?](#)
 - [Which expressions can chain?](#)
 - [Which operator sequences can chain?](#)
 - [Which about conversions or rvalues?](#)
 - [What about fold expressions?](#)
- [Proposal](#)
- [Acknowledgements](#)

Revision History

Since r0. After discussion in Jacksonville:

- Proposed changes to fold-expressions have been removed
- Once we determine that a comparison sequence could chain based on types, the sequence should either chain (based on operator sequence) or be ill-formed. The initial proposal allowed `1 < 4 > 3` to continue to be well-formed (and yield `false`), the new proposal makes it ill-formed.
- A new issue is now discussed regarding conversions and rvalues.

Introduction

The idea of chaining comparisons was first put forth in [P0515R0](#), in section 3.3, reproduced here in its entirety, with a clarifying change to the lambda.

C++17 has added fold expressions, which are very useful. However, as Voutilainen and others have reported, fold expressions do not currently work with comparisons. For example:

```
if (args <...) // not possible with correct semantics in C++17
```

We can permit two-way comparisons to be chained with the usual pairwise mathematical meaning when the mathematical meaning preserves transitivity (which also always means they have equal precedence). The valid chains are:

- all `==`, such as `a == b == c == d;`
- all `{<, <=}`, such as `a < b <= c < d;` and
- all `{>, >=}` (e.g., `a >= b > c > d`).

For example, this:

```
if (a < b <= c < d)
```

would be rewritten by the compiler as-if as follows except with single evaluation of b and c:

```
if ((a < b) && (b <= c) && (c < d)) // but no multiple eval of b and c
```

To illustrate how the compiler would implement this, here is one valid implementation that would satisfy the requirements including single evaluation, by just defining and invoking a lambda:

```
auto __lambda = [&]{
    // a and b both evaluated exactly once
    auto&& __eval_b = b;
    return a < __eval_b && [&]{
        // c only evaluated if a < b. d only evaluated if the first two conditions are true
        auto&& __eval_c = c;
        return __eval_b <= __eval_c &&
            __eval_c < d;
    }();
};
if (__lambda())
```

Chaining support was [one alternative suggested](#) by Ville Voutilainen to permit natural use of comparisons in C++17 fold expressions, such as `if (args <...)`. However, chaining is also broadly useful throughout people's code, so instead of baking the feature into fold expressions only, it's better to provide general-purpose support that can also express concepts like `first <= iter < last`. Providing general chaining also enables fold expressions as a special case (and with the "transitive" restriction above avoids the design pitfall of just providing chaining "for all comparison fold expressions," when they should correctly be supported "for all comparison fold expressions except `!=`" because `!=` is not transitive).

Without chaining, today we either perform double evaluation or introduce a temporary variable. I've many times wanted to write code like `0 <= expr < max` without either evaluating `expr` twice or else having to invent a temporary variable (and usually a new scope) to store the evaluated value. A number of times, I've actually written the code without thinking, forgetting it wasn't supported, and of course it either didn't compile or did the wrong thing. As an example of "did the wrong thing," this proposal does change the meaning of some code like the following that is legal today, but that is dubious because it probably doesn't do what the programmer intended:

```
int x = 1, y = 3, z = 2;
assert (x < y < z); // today, means "if (true < 2)" - succeeds
```

In this proposal, the meaning of the condition would be `if ((1 < 3) && (3 < 2))` and the assertion will fire. To use Stroustrup's term, I consider this "code that deserves to be broken;" the change in meaning is probably fixing a bug. (Unless of course we do a code search and find examples that are actually intended.)

Non-chained uses such as `(a < b == c < d)` keep their existing meaning.

Existing Code in C++

The first question we sought to answer is the last question implied above: How much code exists today that uses chained comparison whose meaning would change in this proposal, and of those cases, how many were intentional (wanted the current semantics and so would be broken by this proposal) or unintentional (compile today, but are bugs and would be silently fixed by this proposal)? Many instances of the latter can be found in questions on StackOverflow [\[1\]](#) [\[2\]](#) [\[3\]](#) [\[4\]](#) [\[5\]](#) [\[6\]](#) [\[7\]](#) [\[8\]](#) [\[9\]](#) [\[10\]](#) [\[11\]](#) ...

To that end, we created a [clang-tidy check](#) for all uses of chained comparison operators, ran it on many open source code bases, and solicited help from the C++ community to run it on their own. The check itself casts an intentionally wide net, matching any instance of `a @ b @ c` for any of the six comparison operators, regardless of the types of these underlying expressions.

Overall, what we found was:

- **Zero** instances of chained arithmetic comparisons that are correct today. That is, intentionally using the current standard behavior.

- Four instances of currently-erroneous arithmetic chaining, of the `assert(0 <= ratio <= 1.0);` variety. These are bugs that compile today but don't do what the programmer intended, but with this proposal would change in meaning to become correct.
- Many instances of using successive comparison operators in DSLs that overloaded these operators to give meaning unrelated to comparisons.

Finding zero instances in many large code bases where the current behavior is intended means this proposal has low negative danger (not a significant breaking change). However, a converse search shows this proposal has existing demand and high positive value: we searched for expressions that would benefit from chaining if it were available (such as `idx >= 0 && idx < max`) and found **a few thousand** instances over just a few code bases. That means that this proposal would allow broad improvements across existing code bases, where linter/tidying tools would be able to suggest rewriting a large number of cases of existing code to be clearer, less brittle, and potentially more efficient (such as suggesting rewriting `idx >= 0 && idx < max` to `0 <= idx < max`, where the former is easy to write incorrectly now or under maintenance, and the latter is both clearer and potentially more efficient because it avoids multiple evaluation of `idx`). It also adds strong justification to pursuing this proposal, because the data show the feature is already needed and its lack is frequently being worked around today by forcing programmers to write more brittle code that is easier to write incorrectly.

Existing Code in Python

While we have no experience with this feature in C++, Python has always supported [chaining comparisons](#):

Unlike C, all comparison operations in Python have the same priority, which is lower than that of any arithmetic, shifting or bitwise operation. Also unlike C, expressions like `a < b < c` have the interpretation that is conventional in mathematics [...]

Comparisons can be chained arbitrarily, e.g., `x < y <= z` is equivalent to `x < y and y <= z`, except that `y` is evaluated only once (but in both cases `z` is not evaluated at all when `x < y` is found to be false).

Formally, if `a`, `b`, `c`, ..., `y`, `z` are expressions and `op1`, `op2`, ..., `opN` are comparison operators, then `a op1 b op2 c ... y opN z` is equivalent to `a op1 b and b op2 c and ... y opN z`, except that each expression is evaluated at most once.

Note that `a op1 b op2 c` doesn't imply any kind of comparison between `a` and `c`, so that, e.g., `x < y > z` is perfectly legal (though perhaps not pretty).

The result is the ability to write natural comparison chains, without having to pairwise break them up with `and`s.

However, as the Python documentation itself points out, C++ has higher precedence for the operators `{>, >=, <, <=}` than for the operators `{==, !=}`. As a result, the expression `a < b == c < d` today is parsed as the possibly-meaningful `(a < b) == (c < d)`, and not the likely meaningless `((a < b) == c) < d`. To interpret it as Python does would involve changing the underlying grammar of C++ and break such code (though we did not find any instances of this kind of mixed comparison, *i.e.* `a < b == c`, in our search).

Issues at Hand

There are several questions that need to be answered about how comparison chaining would work in C++.

- **Which operators?** Only consider chaining builtin comparison operators? Or also overloaded operators?
- **Which expressions?** Only chain if each pairwise expression has type `cv bool`? Or if each pairwise expression has a type that models the `Boolean` concept from the [Ranges TS](#) (which would, notably, include `std::true_type`)? Any type at all?
- **Which chains?** Only chain if either each operator is `==`, if each is in `{<, <=}`, or if each is in `{>, >=}`? Or allow any combination of comparison operators?
- **What about conversions and rvalues?** What do we do in the case where we have a comparison chain in which the constituent expressions are rvalues or the relevant comparison function requires a conversion?
- **What about folds?** What should fold expressions do using a comparison operator?

Regardless of the choice of options, it should be noted that parentheses are significant here. Operator chaining would only apply to unparenthesized expressions. Adding parentheses would be one way of expressing intent. This is the same way that Python behaves today, where `5 > 4 > 3` evaluates to `True` (due to its evaluation as `5 > 4 and 4 > 3`) while `(5 > 4) > 3` evaluates as `False` (due to its evaluation as `True > 3`). If those situations arise where a programmer deliberately wants an unchained comparison, that is available to them with the use of parentheses.

We will take each option separately.

Which operators can chain?

We would prefer to see this apply to all operators, built-in and overloaded. This is different from `&&` and `||`, which change behavior when overloaded because then they don't short-circuit. However, there are many user-defined types for which comparison chaining would have desirable, well-defined behavior (e.g. `std::pair`).

Which expressions can chain?

Why do we need a restriction at all? If we decide to only allow for chaining of builtin-operators, then this question is effectively moot. But once we get into the realm of overloaded operators, there are instances of chaining comparisons on objects where the behavior is decidedly *not* related to comparisons. Examples include [Boost.MultiArray](#):

```
range r6 = -3 <= range().stride(2) < 7; // not intended to be a chained comparison
```

or [Boost.Process](#):

```
bp::spawn(
    master_test_suite().argv[1],
    "test", "--prefix-once", "test",
    bp::std_in < in_buf > fut_in, // not a chained comparison
    bp::std_out > fut,
    io_service,
    ec
);
```

or [Boost.Spirit](#):

```
rule<char const*> r;
r = '(' > int_ > ',' > int_ > ')'; // not a chained comparison
```

or even the less obvious [Catch2](#):

```
std::vector<int> v;
REQUIRE(v.size() == 0); // macro expands to Catch::Decomposer() <= v.size() == 0, not a chained comparison
```

Simply stating that *all* comparison chains get transformed into pairwise comparisons `&&`-ed together would definitely break code. We cannot cast a net that wide.

The simplest approach would be just to accept strictly boolean sub-expressions as candidates. That is, the expression `a @ b @ c` is transformed into `a @ b && b @ c` only if both `a @ b` and `b @ c` have type `cv bool`. This would allow the most typical expected usage of range checking on arithmetic types or equality checking amongst many objects, while also avoiding changing the meanings of any of the above examples. If we allow overloaded operators as well, and those overloaded operators return `bool` (as is typical, and as would be implicitly generated if using [the new operator<=>](#)), then this would already allow for a wide variety of uses.

However, there additionally exists some code that has comparison operators that, rather than returning `bool` instead return `std::true_type` or `std::false_type`. Such return types are common in metaprogramming libraries, where we can encode the result into the type of the return object, instead of just the value. These types do satisfy the `Boolean` concept without being strictly `bool`, and seem safe to be included. Metaprogramming code could benefit from improved readability as well. It seems safe to include this wider range of possible types.

For overloaded comparisons operators that do not return a `Boolean` type, chaining can still be supported but just is not automatic: it is the responsibility of the overloaded operator author to make chaining work correctly for their comparison if that is what they want. We observe that these already exist, where overloaded operators like the `Boost.MultiArray` example already implement a flavor of chaining behavior even in the absence of precedents in the language.

Which operator sequences can chain?

In its original presentation in P0515R0, only a specific subset of comparison operator sequences lead to chaining. Those operator sequences were precisely those that maintain transitivity:

- all `==`, such as `a == b == c == d;`
- all `{<, <=}`, such as `a < b <= c < d;` and
- all `{>, >=}`, such as `a >= b > c > d.`

The ability to chain these operator sequences offers clear improvement to readability in real-world code, including major commercial projects:

(src)	Today	Proposed
clang	<pre>return Success((CR_r == APFloat::cmpEqual && CR_i == APFloat::cmpEqual), E);</pre>	<pre>return Success((CR_r == CR_i == APFloat::cmpEqual), E);</pre>
LLVM.Demangle	<pre>} else if ('1' <= first[1] && first[1] <= '9') {</pre>	<pre>} else if ('1' <= first[1] <= '9') {</pre>
Boost.Numeric	<pre>return x.upper() >= y && y >= x.lower();</pre>	<pre>return x.upper() >= y >= x.lower();</pre>
Boost.Regex	<pre>if(sub < (int)m_subs.size() && (sub > 0))</pre>	<pre>if(0 < sub < (int)m_subs.size())</pre>

The Python language, on the other hand, has no such restrictions. Any comparison operator sequence chains, but this appears to permit mainly pitfalls, not new good uses. In particular, it allows for some reasonable-appearing chains like `a < b == c < d`, but also allows some less likely chains like `a < b > c` and `a != b != c`, which are known pitfalls - the Python documentation has to emphasize that these do not actually imply any relationship between `a` and `c`. We believe that further investigation in analyzing C++ code bases and languages like Python support the position that all of the chains initially recommended in P0515R0 are useful and should be supported, and that all of the chains not recommended in P0515R0 are useless or actively harmful, and so should not be interpreted as chained (any code that writes such chains almost certainly will get something unintended).

We were able to find several expressions of the unrestricted variety that might theoretically be shorted by chaining, but (a) the following rewrites could never actually be made to work without changing the precedence of `==` and `!=` with respect to `<`, `<=`, `>`, and `>=` which would be an impossibly large breaking change to consider, and (b) even if we did that, the resulting code is not actually better. In our opinion, it is visually ambiguous and unclear in all cases. Discussion in Jacksonville indicated that keeping the current unchained behavior for non-transitive comparison sequences would be needlessly confusing as it give too many different potential meanings to an expression. We therefore suggest that the non-transitive comparison sequences be ill-formed:

(src)	Today	Python-like chaining (proposed ill-formed)
Boost.Math	<pre>if((floor(z) == z) && (z < max_factorial::value))</pre>	<pre>if((floor(z) == z < max_factorial::value))</pre>
LLVM.Transforms	<pre>if (ObjectSize == Later.Size && ObjectSize >= Earlier.Size)</pre>	<pre>if (Later.Size == ObjectSize >= Earlier.Size)</pre>
LLVM.Support	<pre>assert(count != 0 && count <= APFloatBase::integerPartWidth / 4);</pre>	<pre>assert(0 != count <= APFloatBase::integerPartWidth / 4);</pre>
LLVM.CodeGen	<pre>assert((LCM >= A && LCM >= B) && "LCM overflow");</pre>	<pre>assert((A <= LCM >= B) && "LCM overflow");</pre>
Boost.Intrusive	<pre>if(n != p && i != p)</pre>	<pre>if(n != p != i)</pre>

What about conversions or rvalues?

An important question issue is how to deal with expressions in which a call to a comparison function would require a user-defined conversions to take place. Consider a piece of code such as:

```
struct X { ... };
bool operator<=(X const&, X const& );
struct Y { operator X() const; };

bool in_between(X const& a, Y const& y, X const& b) {
    // today
    return a <= y && y <= b;

    // with chaining
    return a <= y <= b;
}
```

Today, we have to write `y` twice, so the fact the conversion to `X` is performed twice is unsurprising. But the same two conversions would have to happen in the chained comparison as well, which may be more surprising - as `y` is only referenced once.

An additional question is what do we do if one of the comparison functions actually would move from its arguments?

```
struct A { ... }; // imagine this has a move constructor that modifies its source
bool operator<(A, A);

A mid();
bool in_between(A const& left, A const& right) {
    return left < mid() < right;
}
```

If `A` both (a) has a non-trivial move constructor and (b) has comparisons that take by value, then this could move from `mid()`'s returned object twice. But frankly, we believe that this is just plain weird, for two reasons. First, comparisons are logically const operations and should never change the values of their arguments; a comparison that could move from its argument in a way that modifies the source's value is nonsensical. Second, a type that is designed to be passed by value to comparisons or other functions (such as `int` or `std::string_view`) is naturally a cheap-to-copy value type that does not have a move constructor that modifies the source object (or indeed, does something different from copying). We cannot come up with a defensible example where this would be a problem, but we would invite anyone to show a plausible example of such code.

This leads to the question of what to do with with middle arguments that are rvalues. We have three options:

1. After evaluating the middle argument, we forward it to both sides of the comparison:

```
auto&& __mid = mid();
return left < __FWD(mid) && __FWD(mid) < right;
```

This leads us to moving from that `A` twice, which is a non-starter.

2. We only forward at most one time:

```
auto&& __mid = mid();
return left < mid && __FWD(mid) < right;
```

This means we're treating the *same* expression in *two different* ways, even though it's only named once. This seems unnecessarily difficult to reason about, and we're not sure it would provide commensurate benefit.

3. We just treat all the expressions as lvalues, always materializing a temporary and using it:

```
auto&& __mid = mid();
return left < __mid && __mid < right;
```

We believe the third option is the only viable option. Any kind of attempt at restricting rvalue expressions in the middle of chain would restrict useful comparisons. The implication is that expressions that look like rvalues would be treated as lvalues, so that expressions that look like they should lead to moves may actually lead to copies, and expressions that are named once could still lead to two conversions. We're not concerned about the first problem since, as described above, we expect that comparison functions either take

their arguments by value and are cheap to copy (so that there simply is no difference between incurring a move and incurring a copy) or take their arguments by reference to const (so that neither a move nor a copy happens to begin with).

And we could do something better about the second problem. We could allow, or even mandate (or allow now and mandate in the future), the optimization that if both uses of the duplicated expression invoke the same conversion (whether converting constructor or conversion function) to match the selected operators' respective parameters, we materialize and reuse the converted result. This has precedent in copy elision and guaranteed copy elision.

That is, the following example:

```
struct X { ... };
bool operator<=(X const&, X const& );
struct Y { operator X() const; };
Y foo();

bool in_between(X const& a, X const& b) {
    return a <= foo() <= b;
}
```

would behave as if:

```
bool in_between(X const& a, X const& b) {
    auto&& __materialized_foo = foo();
    auto&& __converted_foo = __materialized_foo.operator X();

    return a <= __converted_foo && __converted_foo <= b;
}
```

This lets us have multiple benefits: we can chain the comparison instead of splitting it up, the middle expressions are only evaluated (at most) one time, and the necessary conversions are only evaluated (at most) one time as well.

What about fold expressions?

Today, all six of the comparison operators are valid binary operators to use in a [fold expression](#), but the expansion rules always produce [parenthesized sub-expressions](#). That is:

```
template <typename... Ts>
bool ordered(Ts... vals) {
    return (... <= vals);
}

ordered(4, 3, 2, 1); // instantiated as ((4 <= 3) <= 2) <= 1, evaluates to true, even with this proposal
```

As mentioned earlier, parentheses are significant, so having fold expressions expand as parenthesized would continue to inhibit comparison chaining. This makes today's fold expressions for comparisons not useful and actually buggy.

The initial version of this proposal suggested that fold expressions over comparison operators instantiate as unparenthesized, to allow them to be useful and correct. However, parenthesized sub-expressions are a key part of how fold expressions actually work. It would be a large change, both conceptually and in terms of implementation, to remove them - a change that fails to offer comparable large benefit. We therefore propose no change to fold expressions at this time.

Proposal

A maximal, unparenthesized expression of the form `a @1 b @2 c @3 ... @# n`, where each `@` is one of the six two-way comparison operators, that contains at least two comparison operators will now be evaluated according to the following rules:

- If each pairwise comparison (that is `a @1 b`, `b @2 c`, etc.) would evaluate to a type that models the `Boolean` concept, then:
 - If it is not the case that every operator is either `==`, or one of `{<, <=}`, or one of `{>, >=}` (that is, the sequence of operators would not be transitive), then the expression is ill-formed.

- Otherwise, the expression shall be evaluated as if `(a @1 b) && (b @2 c) && ... && (m @# n)`, except that each sub-expression shall be evaluated at most once time (including temporary materialization) and each of the "middle" terms will always be passed to the corresponding comparison functions as lvalues. If any of these "middle" terms need to undergo the same conversion to match its two selected operators' respective parameters, an implementation may elide the second conversion and reuse the same converted value as the argument for both parameters.

In other words, approximately equivalently to:

```
[&]{
    auto&& __eval_b = b;
    return (a @1 __eval_b) && [&]{
        auto&& __eval_c = c;
        return (__eval_b @2 __eval_c) && [&]{
            // ...
        };
    };
}();
}()
```

- Otherwise, the behavior remains as it does today.

Any DSLs currently existing would continue to work properly, since the results of each sub-expression would not model `Boolean`. The non-transitive chains (such as `a != b != c`) would be made ill-formed by the first sub-bullet. If non-transitive comparisons are required, parentheses may be used to break up sub-expressions to avoid this rule (e.g. `(a < b) == (c < d)` would behave as it does today).

The second sub-bullet, the as-lvalue rule, provides a path to allow chained comparisons with `string_view` and `string`, other rvalues or conversions, as well as providing an easy rule to remember. From the conversion section, `a <= y <= b` would be a valid comparison chain that would evaluate as `auto&& __y = y; (a <= __y) && (__y <= b)`, which would lead to one invocation of `Y::operator X() const` (the second would be allowed to be elided).

Acknowledgements

Thanks to T.C. for help in properly specifying the search for chained comparisons and innumerable feedback about various issues. Thanks to Nicolas Lesser and Titus Winters for contributing data for live use of comparisons.